

Kalman filter for INS/IMU - Group room exercise

Torben Knudsen

1 Introduction

This exercise is for a half day in the course Sensors and Systems. The purpose is to use Kalman filtering technique with the sensors on the Arduino. To keep it simple the build in accelerometer is used in one dimension. The Arduino is placed in a know start position and moved by hand forth and back to end at the start position. Methods to estimate the speed and position from the acceleration measurement can then be used. The start and end position and speed is known to be zero and they will vary up and down in between. This is what can be used to access the methods. The simple method is to integrate the acceleration measurements one time for speed estimate and a second time for position estimate. This can serve as a benchmark for the Kalman filter method.

Notice that neither speed nor position are measured. Only acceleration is measured with noise and bias. This means that estimation of acceleration can be expected to perform well. Depending on the models used, estimates of speed and position will easily be drifting away as they are mainly integration of the acceleration.

Further the Arduino is moved by hand so only a generic dynamic model can be used. In this way this problem is more difficult compared to the ideal case for a KF where the model is well known and the state is fully observable.

2 Problem

Given a known initial condition where position, speed and acceleration is zero, estimate the acceleration, speed and position from acceleration measurements. Assume the Arduino is moved by hand in the x dimension only. Initially no time for calibration is assumed.

After having investigated the above problem there is endless possibilities for more advanced problems by increasing the dimension to 2 and 3, including more or all accelerometer measurements and gyro measurements plus maybe external sensors e.g. GPS or distance.

3 Sensor investigation

Even though the problem does not assume a initial calibration every time the estimation is started it is assumed that one sensor analysis can be done.

Place the Arduino somewhere as steady and horizontal as possible and measure at 13Hz for a minute or so. Estimate the bias, standard deviation, min and max plus maybe spectra. Maybe you have already

done this. In that case you can redo it easily or use the already gained results.

The Moodle page includes a template for changing the sample rate of the Arduino accelerometer and also for doing the analysis of the data for a fixed position.

Discuss what the results can be used for in the development of the Kalman filter. Assume the size of the standard deviation is representative and can be used but the bias is individual for the particular Arduino.

One way to handle the placement and movement of the Arduino on the table is to place it in the foam it was delivered with so the foam with the Arduino can be slid over the table.

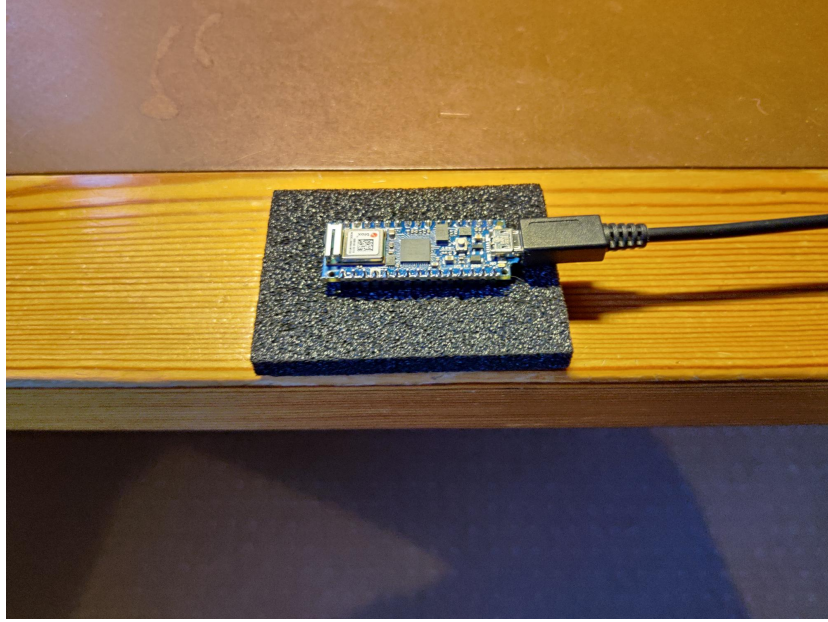


Figure 1: Arduino on foam for easier sliding.

4 Benchmark method

The benchmark method consist in integrating the acceleration measurement ones and twice to obtain the velocity and position according to the kinematics. Using a standard state space formulation this is given by (1). From a mathematical point of view this will give correct results. However, any small errors will spoil this.

$$\begin{pmatrix} \dot{v} \\ \dot{p} \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} v \\ p \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix} a \quad (1a)$$

$$v(0) = v_0, \quad p(0) = p_0 \quad (1b)$$

The KF estimation algorithm should at least perform as well as the benchmark method. If it works better it can be considered a success.

5 Model

The Kalman filter consist of a state model describing the dynamics and a measurement model that relates the measurement to the state.

5.1 State model

As the measurement is the acceleration a obvious model has a 3 dimensional state consisting of acceleration, velocity and position. As the force on the Arduino is unknown a first order lowpass filter is used to model slowly varying forces.

The real world physics and kinematics develop in continuous time. A exact method for transforming from continuous to discrete time is briefly explained in appendix B but it is not part of the course as it rely on stochastic differential equations (SDE).

Using the forward Euler method as in (3) is and approximation which normally works sufficiently well when the sampling time is sufficiently small.

Modeling the acceleration a as lowpass filtered white noise is done using the AR(1) process

$$a_{k+1} = \phi a_k + w_{a_k}, \quad w_a \in \text{NID}(0, \sigma_{q_a}^2) \Rightarrow \quad (2a)$$

$$\sigma_a^2 = \frac{\sigma_{q_a}^2}{1 - \phi^2} \quad (2b)$$

If the desired bandwidth for the acceleration is ω_b , ϕ is given by $\phi = e^{-\omega_b T_s}$. By deciding on σ_a , σ_{q_a} can be found using (2b).

The kinematic part can be approximated using forward Euler as

$$v_{k+1} = v_k + a_k T_s \quad (3a)$$

$$p_{k+1} = p_k + v_k T_s \quad (3b)$$

The model can be extended with a acceleration measurement bias model. If a slowly varying bias is assumed the extra bias state can be modeled by the random walk

$$b_{k+1} = b_k + w_{b_k}, \quad w_b \in \text{NID}(0, \sigma_{q_b}^2), \quad (4a)$$

$$b_0 = 0, \quad \text{Cov}(b_0) = P_{b0} \quad (4b)$$

Collecting all the above the simple discrete time model becomes

$$x_{k+1} = \Phi x_k + w_k, \quad w_k \in \text{NID}(\underline{0}, Q) \quad (5a)$$

$$x = \begin{pmatrix} a \\ v \\ p \\ b \end{pmatrix}, \quad \Phi = \begin{pmatrix} \phi & 0 & 0 & 0 \\ T_s & 1 & 0 & 0 \\ 0 & T_s & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad Q = \begin{pmatrix} \sigma_{q_a}^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \sigma_{q_b}^2 \end{pmatrix}, \quad (5b)$$

$$x_0 = \underline{0}, \quad \text{Cov}(x_0) = P_0 \quad (5c)$$

5.2 Measurement model

The measurement including the bias is given below.

$$y_k = Hx_k + v_k, \quad H = \begin{pmatrix} 1 & 0 & 0 & 1 \end{pmatrix}, \quad v_k \in \text{NID}(0, R) \quad (6)$$

The measurement model without the bias is

$$y_k = Hx_k + v_k, \quad H = \begin{pmatrix} 1 & 0 & 0 \end{pmatrix}, \quad x = \begin{pmatrix} a & v & p \end{pmatrix}^T \quad (7)$$

6 Kalman filter

As all the above models are linear the standard Kalman filter can be used. The KF algorithm can be found for example in slide 23 from the first estimation lecture (lecture 6). Also the program KF.m from lecture 7 can be used as a template.

If you need the EKF the algorithm is in slide number 7 in EKFAAndUKFHandout.pdf from Lecture 7 and a template is found in EKF.m in the exercise from Lecture 7.

7 Tasks

The measurements must be collected using a suitable Arduino program. One option is then to load the collected data into matlab and do all the estimation here. Another option is to do some or all the estimation on the Arduino and collect the results and show them on the computer somehow. The split between matlab and the Arduino is up to you. It is also possible to start with matlab and move to the Arduino if you have time for it.

7.1 Make the experiment

Place the Arduino on a corner of the table and move it smoothly forth and back a couple of times as far as the wire allows.

7.2 Direct kinematics method

Calculate the velocity and positions using the direct kinematic methods in (1). Depending on the particular Arduino and bias the results can be poor.

7.3 Basic Kalman filter

Program a standard KF based on (8) but without the bias state. Discuss how to choose all the parameters.

1. How can you suggest discrete time parameters ϕ, σ_{q_a} for the acceleration part using (2)?
2. The measurement noise R can be chosen according to the variance and bias estimated in section 3.

3. The initial state is set to zero. The initial covariance can be chosen as a diagonal matrix with small or zero values.
4. Plot the state estimates together with ± 2 standard deviations.
 - The real acceleration and speed is not known but the position is known to be between start and end point.
 - Especially the start and end state is known to be zero.
 - Does the real speed and position state seem to be inside the confidence interval?

Expect the results for speed and position to be poor. Is it possible to recognize the movement in speed and position? How does the results compare to the direct kinematics method?

7.4 Include estimation of bias

Include the bias state in the model and Kalman filter. Choose the initial mean bias $b_0 = 0$. Moreover choose the whole initial state mean to zero i.e. $x_0 = \underline{0}$. Start by choosing the initial covariance for acceleration, speed and position to zero i.e. $P_{0_{1:3,1:3}} = \underline{0}$.

1. What can be used for suggesting the initial bias variance $P_{b0} = P_{0_{4,4}}$?
2. Is there any reason to chose the measurement noise variance R differently in this case compared to the one without the bias state included?
3. Plot the state estimates together with ± 2 standard deviations.
 - How does the bias state estimate develop?
 - Can you explain this?
 - Can you chance this?
 - By changing the parameters already in play?
 - Could there be any reason for changing the bias state model (4a)?
 - Does the real speed and position state seem to be inside the confidence interval?
4. Does it make a difference if you choose a non zero initial covariance e.g. $P_{0_{1:3,1:3}} = I$? Explain why?

Is it possible to recognize the movement in speed and position? How does the results compare to the direct kinematics method and the Kalman filter not including the bias state?

7.5 Further work

As already stated in section 2 there are plenty of additional investigations to be done based on the above setup.

- For 1D more sophisticated modeling of the acceleration and bias can be developed and tested see e.g. appendix A.

- 2D movements. Then acceleration in two directions e.g. x, y and the rotation in one direction z is required. This will give a nonlinear model.
- 3D movements. Then accelerations and rotations in all three directions are needed. This will give a very nonlinear model. Also it is quite complicated due to the 3D dynamics involving quaternions or similar.
- More sensors can/should be included to obtain “observable” speed and position states.
 - Magnetometer
 - Distance to fixed points
 - GPS

A Stable kinematic models

The below is not part of the curriculum for the course but is meant as inspiration to how much can be done with the model part of estimation.

The model (8) is a natural starting point as the kinematics is included and the unknown acceleration are modeled as a AR(1) process with zero mean and suitable bandwidth and variance. However, the model is not stable as it includes the two integrators and no feedback. This means that the variance of velocity and position goes to infinity with time. This is not realistic. Can the model be changed to get a limited variance on position and velocity. The answer is yes. One approach is to use feedback to design such a model.

Start with a model that has constant acceleration (or integrated white noise if process noise is assumed) and where velocity is integrated acceleration and position is integrated velocity then also assume that the feedback is from all states but only to the acceleration in order not to spoil the kinematics. This gives

$$x_{k+1} = Ax_k + Bu_k \quad (8a)$$

$$x = \begin{pmatrix} a \\ v \\ p \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 0 & 0 \\ T_s & 1 & 0 \\ 0 & T_s & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad (8b)$$

$$u = Kx \quad (8c)$$

To get a suitable dynamics i.e. frequency contents K can be found tuning R and using LQR below

<code>S= ss(A,B,eye(3),0,Ts);</code>	<code>% The C matrix has noeffect</code>
<code>Q= diag(ones(3,1));</code>	<code>% Equal weight on states</code>
<code>R= 100^2;</code>	<code>% Tuning the dynamics</code>
<code>[K,S,P]= lqr(S,Q,R);</code>	

To get a suitable variance e.g. one on the position the Lyapunov equation can be solved using DLYAP below

<code>% For tuning the variance of states</code>	
<code>Q= diag([1^2 0 0])</code>	<code>% Unit variance process noise</code>
<code>P= dlyap(A-B*K,Q)</code>	
<code>% To get position variance 1 choose process noise 1/(P(3,3))</code>	

$$\begin{aligned} Q &= \text{diag}([1^2 \ 0 \ 0])/P(3,3) \\ P &= \text{dlyap}(A-B*K,Q) \end{aligned}$$

This gives the (closed loop) model

$$x_{k+1} = A_{cl}x_k + w_k, \quad w_k \in \text{NID}(\underline{0}, Q) \quad (9a)$$

$$x = \begin{pmatrix} a \\ v \\ p \end{pmatrix}, \quad T_s = 1/13, \quad (9b)$$

$$A_{cl} = (A - BK) = \begin{pmatrix} 0.9197 & -0.0408 & -0.009606 \\ T_s & 1 & 0 \\ 0 & T_s & 1 \end{pmatrix}, \quad Q = \begin{pmatrix} 0.0076152 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad (9c)$$

$$\text{Cov}(x) = P = \begin{pmatrix} 0.066824 & -0.0025702 & -0.12429 \\ -0.0025702 & 0.12449 & -0.0047881 \\ -0.12429 & -0.0047881 & 1 \end{pmatrix} \quad (9d)$$

B Using continuous time and sampling

The real world physics and kinematics develop in continuous time. Transforming from continuous to discrete time using the forward Euler method as in (3) is an approximation which normally works sufficiently well. However, it is safer to use the exact discretization method that exist for linear systems. I assume you have not learned this method as it rely on stochastic differential equations. It is briefly explained below but it is not part of the course.

The continuous time stochastic differential equation (SDE) (10a) has a mean value given by (10b) and a covariance given by (10c). The stochastic input w is a Wiener process with incremental covariance S which means that $\text{Cov}(w(t + \delta t) - w(t)) = S\delta t$.

$$dx = Fxdt + Gdw, \quad w \in W(S) \Rightarrow \quad (10a)$$

$$\dot{\mu} = F\mu, \quad \mu(t) \triangleq E(x(t)) \quad (10b)$$

$$\dot{P} = FP + PF^T + GSG^T, \quad P(t) \triangleq \text{Cov}(x(t)) \quad (10c)$$

The discretized Markov model is given by (11) where T_s is the sample time. Notice that Φ is the same as in the deterministic case. $P(T_s)$ can be found from (10c) using initial condition $P(0) = \underline{0}$ either analytical or by numerical integration. A matlab function for this is on the Moodle page.

$$x_{k+1} = \Phi x_k + w_k, \quad w_k \in \text{NID}(\underline{0}, Q), \quad (11a)$$

$$\Phi = e^{FT_s}, \quad Q = P(T_s), \quad P(0) = \underline{0} \quad (11b)$$

The acceleration part of the SDE can be modeled similar to (2) as

$$da = -\omega_b a dt + dw_a, \quad w_a \in W(s_a) \Rightarrow \sigma_a^2 = \frac{s_a}{2\omega_b} \quad (12)$$

where again ω_b and σ_a are the decided bandwidth and standard deviation for the acceleration.

As an example the case with constant bias would be given by

$$dx = Fxdt + dw, \quad w \in W(S), \quad (13a)$$

$$F = \begin{pmatrix} -\omega_b & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad (13b)$$

$$S = \begin{pmatrix} s_a & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad (13c)$$

The differences can be seen in the below numerical example where $\omega_b = 0.1 \cdot 2\pi$, $\sigma_a = 1$. The forward Euler discretization gives

$$\Phi = \begin{pmatrix} 0.95167 & 0 & 0 & 0 \\ 0.076923 & 1 & 0 & 0 \\ 0 & 0.076923 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad Q = \begin{pmatrix} 0.092139 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2.3077e-05 \end{pmatrix} \quad (14)$$

and the exact method gives

$$\Phi = \begin{pmatrix} 0.95282 & 0 & 0 & 0 \\ 0.075094 & 1 & 0 & 0 \\ 0.0029115 & 0.076923 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad (15a)$$

$$Q = \begin{pmatrix} 0.092182 & 0.003511 & 8.8262e-05 & 0 \\ 0.003511 & 0.00017861 & 5.0187e-06 & 0 \\ 8.8262e-05 & 5.0187e-06 & 1.4905e-07 & 0 \\ 0 & 0 & 0 & 2.3077e-05 \end{pmatrix} \quad (15b)$$

The differences are generally small. Notice though that using the exact method the effect in the acceleration state spreads to the speed and position state during the sampling time even when it is small. This is why there are non zero elements in $\Phi_{3,1}$ and in the off diagonal for Q .